COMPUTER ARCHITECTURE AND ORGANIZATION

AFIT, KADUNA LECTURE NOTE

ARITHMETIC & LOGIC SERIES

(Number System, Codes and Addressing Modes)

JUNE 2021

1.0 Number System

If base or radix of a number system is 'r', then the numbers present in that number system are ranging from zero to r-1. The total numbers present in that number system is 'r'. So, we will get various number systems, by choosing the values of radix as greater than or equal to two.

The following number systems are the most commonly used.

Decimal Number system
Binary Number system
Octal Number system
Hexadecimal Number system

1.1 Decimal Number System

In everyday life, we use a system based on decimal digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) torepresent numbers, and refer to the system as the decimal system. This is probably due to the fact that human has 10 fingers or toes.

The number 4728 means four thousands, seven hundreds, two tens, plus eight, For example:

$4728 = (4 * 1000) + (7 * 100) + (2 * 10) + 8$

The decimal system is said to have a base, or radix, of 10. This means that each digitin the number is multiplied by 10 raised to a power corresponding to that digit's position:

$83 = (8 * 10^1) + (3 * 10^0)$

$4728 = (4 * 10^3) + (7 * 10^2) + (2 * 10^1) + (8 * 10^0)$

The same principle holds for decimal fractions, but negative powers of 10 are used. Thus, the decimal fraction 0.256 stands for 2 tenths plus 5 hundredths plus 6 thousandths:

$0.256 = (2 * 10^{-1}) + (5 * 10^{-2}) + (6 * 10^{-3})$

A number with both an integer and fractional part has digits raised to both positive and negative powers of 10:

$442.256 = (4 * 10^2) + (4 + 10^1) + (2 * 10^0) + (2 * 10^{-1}) + (5 * 10^{-2})$

$+ (6 * 10^{-3})$

The decimal number system is a *positional number system*.
In a positional number system, each number is represented by a string of digits inwhich each digit position i has an associated weight r^i, where r is the radix, or base,of the number system.

Table 1.1(a) & (b) shows the relationship between each digit position and the value assigned to that position. Each position is weighted 10 times the value of the position to the right and one-tenth the value of the position to the left. Thus, positions represent successive powers of 10. If we number the positions as indicated in Table 1.1(a), then position i is weighted by the value 10^i

Table 1.1(a): Positional Interpretation of a Decimal Number

4	7	2	2	5	6
100s	10s	1s	tenths	hundredths	thousandths
10^2	10^1	10^0	10^{-1}	10^{-2}	10^{-3}
position 2	position 1	position 0	position –1	position –2	position –3

Table 1.1(b): Positional Interpretation of a Number in Base 7

Position	4	3	2	1	0	–1
Value in Exponential Form	7^4	7^3	7^2	7^1	7^0	7^{-1}
Decimal Value	2401	343	49	7	1	1/7

1.1.1 The Binary System

All digital circuits and systems use this binary number system. The base or radix of this number system is 2. So, the numbers 0 and 1 are used in this number system.

The part of the number, which lies to the left of the binary point is known as integer part. Similarly, the part of the number, which lies to the right of the binary point is known as fractional part.

In this number system, the successive positions to the left of the binary point having weights of 2^0, 2^1, 2^2, 2^3 and so on. Similarly, the successive positions to the right of the binary point having weights of 2^{-1}, 2^{-2}, 2^{-3} and so on. That means, each position has specific weight, which is power of base 2.

Example 1

Consider the binary number 1101.011. Integer part of this number is 1101 and fractional part of this number is 0.011. The digits 1, 0, 1 and 1 of integer part have weights of 2^0, 2^1, 2^2, 2^3 respectively. Similarly, the digits 0, 1 and 1 of fractional part have weights of 2^{-1}, 2^{-2}, 2^{-3} respectively.

Mathematically, we can write it as:
1101.011 = (1 × 23) + (1 × 22) + (0 × 21) + (1 × 20) + (0 × 2-1) + (1 × 2-2) + (1 × 2-3)

After simplifying the right hand side terms, we will get a decimal number, which is an equivalent of binary number on left hand side.

1.1.2 CONVERTING BETWEEN BINARY AND DECIMAL

(a) Decimal Number to other Bases Conversion

If the decimal number contains both integer part and fractional part, then convert both the parts of decimal number into other base individually. Follow these steps for converting the decimal number into its equivalent number of any base 'r'.

i. Do division of integer part of decimal number and successive quotients with base 'r' and note down the remainders till the quotient is zero. Consider the remainders in reverse order to get the integer part of equivalent number of base 'r'. That means, first and last remainders denote the least significant digit and most significant digit respectively.

ii. Do multiplication of fractional part of decimal number and successive fractions with base 'r' and note down the carry till the result is zero or the desired number of equivalent digits is obtained. Consider the normal sequence of carry in order to get the fractional part of equivalent number of base 'r'.

(b) Decimal to Binary Conversion

The following two types of operations take place, while converting decimal number into its equivalent binary number.

i. Division of integer part and successive quotients with base 2.
ii. Multiplication of fractional part and successive fractions with base 2.

Example 2
Consider the decimal number 58.25. Here, the integer part is 58 and fractional part is 0.25.

Step 1 – Division of 58 and successive quotients with base 2.

Operation	Quotient	Remainder
58/2	29	0 LSB
29/2	14	1
14/2	7	0
7/2	3	1
3/2	1	1
1/2	0	1 MSB

$\Rightarrow 58_{10} = 111010_2$

Therefore, the integer part of equivalent binary number is 111010.

Step 2 – Multiplication of 0.25 and successive fractions with base 2.

Operation	Result	Carry
0.25 x 2	0.5	0
0.5 x 2	1.0	1
-	0.0	-

$$\Rightarrow .25_{10} = .01_2$$

Therefore, the fractional part of equivalent binary number is .01

$\Rightarrow 58.25_{10} = 111010.01_2$

Therefore, the binary equivalent of decimal number 58.25 is 111010.01.

(c) Binary Number to other Bases Conversion

The process of converting a number from binary to decimal is different to the process of converting a binary number to other bases.

Binary to Decimal Conversion

For converting a binary number into its equivalent decimal number, first multiply the bits of binary number with the respective positional weights and then add all those products.

Example 3

Consider the binary number 1101.11.

Mathematically, we can write it as
$1101.11_2 = (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) + (1 \times 2^{-2})$

$\Rightarrow 1101.112 = 8 + 4 + 0 + 1 + 0.5 + 0.25 = 13.75$

$\Rightarrow 1101.11_2 = 13.75_{10}$

Therefore, the decimal equivalent of binary number 1101.11 is 13.75.

(d) Binary to Hexa-Decimal Conversion

We know that the bases of binary and Hexa-decimal number systems are 2 and 16 respectively. Four bits of binary number is equivalent to one Hexa-decimal digit, since $2^4 = 16$.

Follow these two steps for converting a binary number into its equivalent Hexa-decimal number.

i. Start from the binary point and make the groups of 4 bits on both sides of binary point. If some bits are less while making the group of 4 bits, then include required number of zeros on extreme sides.
ii. Write the Hexa-decimal digits corresponding to each group of 4 bits.

Example 4

Consider the binary number 101110.01101 to hexadecimal
Step 1 – Make the groups of 4 bits on both sides of binary point.

10 1110.0110 1

Here, the first group is having only 2 bits. So, include two zeros on extreme side in order to make it as group of 4 bits. Similarly, include three zeros on extreme side in order to make the last group also as group of 4 bits.

⇒ 0010 1110.0110 1000

Step 2 – Write the Hexa-decimal digits corresponding to each group of 4 bits.

$\Rightarrow 00101110.01101000_2 = 2E.68_{16}$

Therefore, the Hexa-decimal equivalent of binary number 101110.01101 is 2E.68

(e) Hexa-Decimal to Binary Conversion

The process of converting Hexa-decimal number into its equivalent binary number is just opposite to that of binary to Hexa-decimal conversion. By representing each Hexa-decimal digit with 4 bits, we will get the equivalent binary number.

Example 5

Consider the Hexa-decimal number 65.4C
Represent each Hexa-decimal digit with 4 bits.

$65.4C_{16} = 01100101.01001100_2$

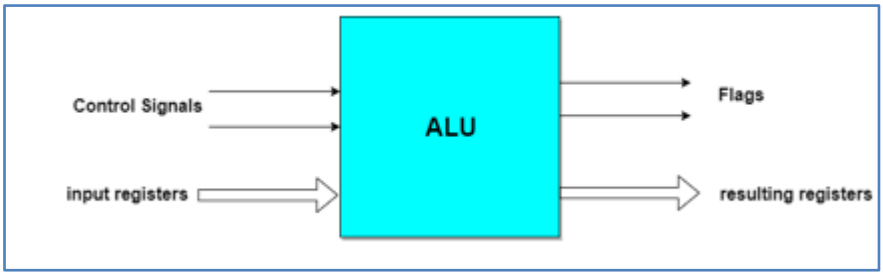
The value does not change by removing the zeros, which are at two extreme sides.

$\Rightarrow 65.4C_{16} = 1100101.010011_2$

Therefore, the binary equivalent of Hexa-decimal number 65.4C is 1100101.010011.

2.0 Arithmetic Logical Unit

Instead of having individual registers performing the micro-operations, computer system provides a number of registers connected to a common unit called Arithmetic Logical Unit (ALU). ALU is the main and one of the most important unit inside CPU of computer. All the logical and mathematical operations of computer are performed here. The contents of specific register is placed in the input of ALU. ALU performs the given operation and then transfer it to the destination register.



2.1 Instruction Codes

While a Program, as we all know, is, a set of instructions that specify the operations, operands, and the sequence by which processing has to occur. An instruction code is a group of bits that tells the computer to perform a specific operation part.

Instruction Code: **Operation Code**

The operation code of an instruction is a group of bits that define operations such as add, subtract, multiply, shift and compliment. The number of bits *required* for the operation code depends upon the total number of operations available on the computer. The operation code must consist of at least n bits for a given 2^n operations. The operation part of an instruction code specifies the operation to be performed.

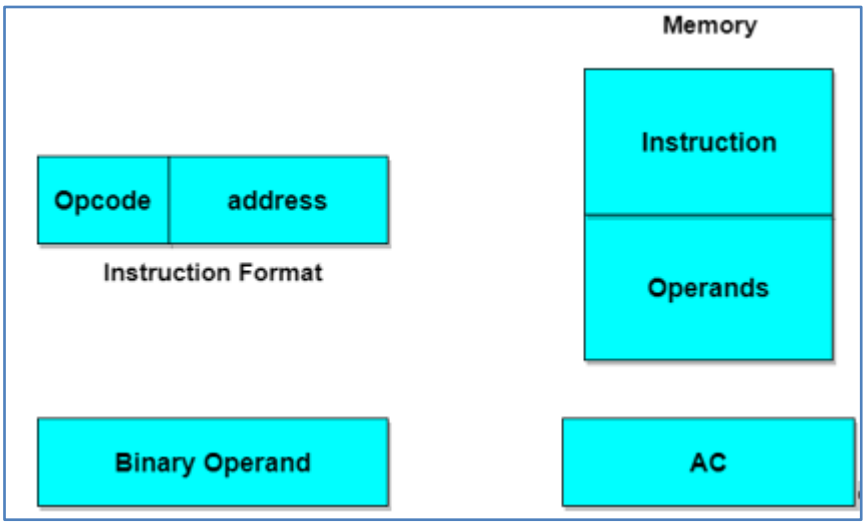
Instruction Code: ***Register Part***

The operation must be performed on the data stored in registers. An instruction code therefore specifies not only operations to be performed but also the registers where the operands (data) will be found as well as the registers where the result has to be stored.

2.2 Stored Program Organisation

The simplest way to organize a computer is to have Processor Register and instruction code with two parts. The first part specifies the operation to be performed and second specifies an address. The memory address tells where the operand in memory will be found.

Instructions are stored in one section of memory and data in another.



Computers with a single processor register is known as Accumulator (ACC). The operation is performed with the memory operand and the content of ACC.

2.2.1 Common Bus System

The basic computer has 8 registers, a memory unit and a control unit. Paths must be provided to transfer data from one register to another. An efficient method for transferring data in a system is to use a Common Bus System. The output of registers and memory are connected to the common bus.

Load (LD)

The lines from the common bus are connected to the inputs of each register and data inputs of memory. The particular register whose LD input is enabled receives the data from the bus during the next clock pulse transition.

Before studying about instruction formats lets first study about the operand address parts.

When the 2nd part of an instruction code specifies the operand, the instruction is said to have immediate operand. In addition, when the 2nd part of the instruction code specifies the address of an operand, the instruction is said to have a direct address. And in indirect address, the 2nd part of instruction code, specifies the address of a memory word in which the address of the operand is found.

2.2.2 Computer Instructions

The basic computer has three instruction code formats. The Operation code (opcode) part of the instruction contains 3 bits and remaining 13 bits depends upon the operation code encountered.

There are three types of formats:

1. Memory Reference Instruction

It uses 12 bits to specify the address and 1 bit to specify the addressing mode (I). I is equal to 0 for direct address and 1 for indirect address.

2. Register Reference Instruction

These instructions are recognized by the opcode 111 with a 0 in the left most bit of instruction. The other 12 bits specify the operation to be executed.

3. Input-Output Instruction

These instructions are recognized by the operation code 111 with a 1 in the left most bit of instruction. The remaining 12 bits are used to specify the input-output operation.

Format of Instruction

The format of an instruction is depicted in a rectangular box symbolizing the bits of an instruction. Basic fields of an instruction format are given below:

1. An operation code field that specifies the operation to be performed.

2. An address field that designates the memory address or register.

3. A mode field that specifies the way the operand of effective address is determined.

Computers may have instructions of different lengths containing varying number of addresses. The number of address field in the instruction format depends upon the internal organization of its registers.

2.2.3 Addressing Modes and Instruction Cycle

The operation field of an instruction specifies the operation to be performed. This operation will be executed on some data that is stored in computer registers or the main memory. The way any operand is selected during the program execution is dependent on the addressing mode of the instruction. The purpose of using addressing modes is as follows:

1. To give the programming versatility to the user.

2. To reduce the number of bits in addressing field of instruction.

Types of Addressing Modes

Below are discussed different types of addressing modes one by one:

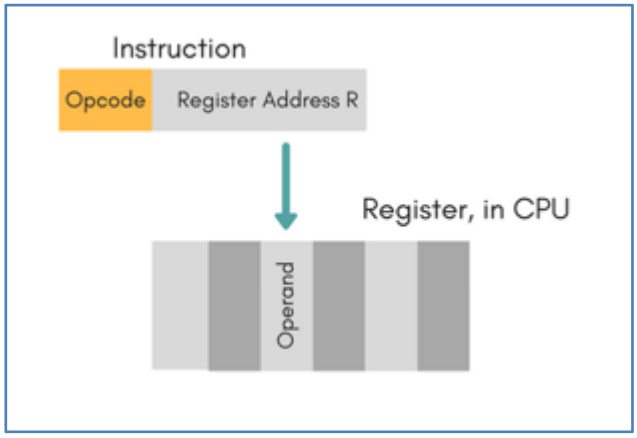
(a) Immediate Mode

In this mode, the operand is specified in the instruction itself. An immediate mode instruction has an operand field rather than the address field.

For example: **ADD 7**, which says Add 7 to contents of accumulator. 7 is the operand here.

(b) Register Mode

In this mode, the operand is stored in the register and this register is present in CPU. The instruction has the address of the Register where the operand is stored.



Advantages

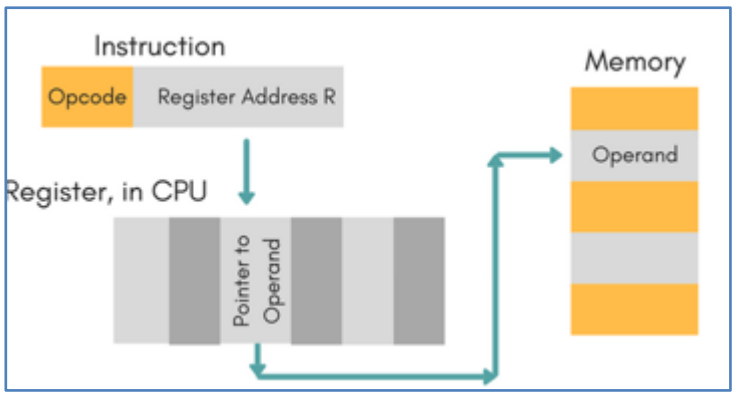
- Shorter instructions and faster instruction fetch.
- Faster memory access to the operand(s)

Disadvantages

- Very limited address space
- Using multiple registers helps performance but it complicates the instructions.

(c) Register Indirect Mode

In this mode, the instruction specifies the register whose contents give us the address of operand which is in memory. Thus, the register contains the address of operand rather than the operand itself.



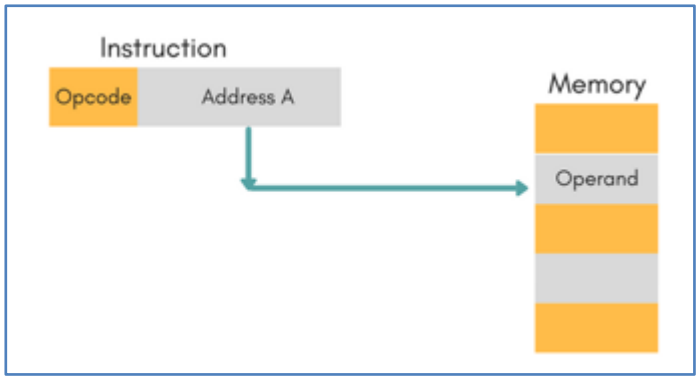
(d) Auto Increment/Decrement Mode

In this the register is incremented or decremented after or before its value is used.

(e) `Direct Addressing Mode

In this mode, effective address of operand is present in instruction itself.

- Single memory reference to access data.
- No additional calculations to find the effective address of the operand.

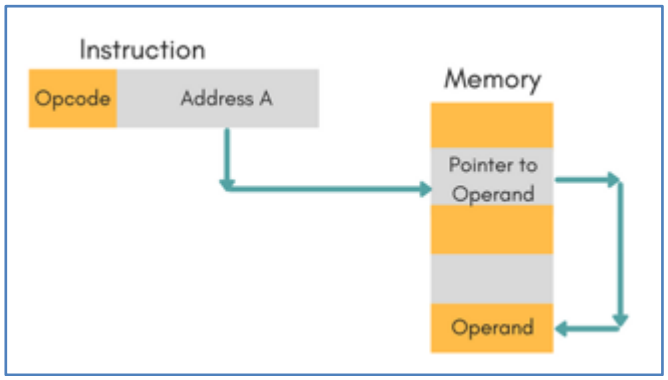


For Example: ADD R1, 4000 - In this the 4000 is effective address of operand.

NOTE: Effective Address is the location where operand is present.

(f) Indirect Addressing Mode

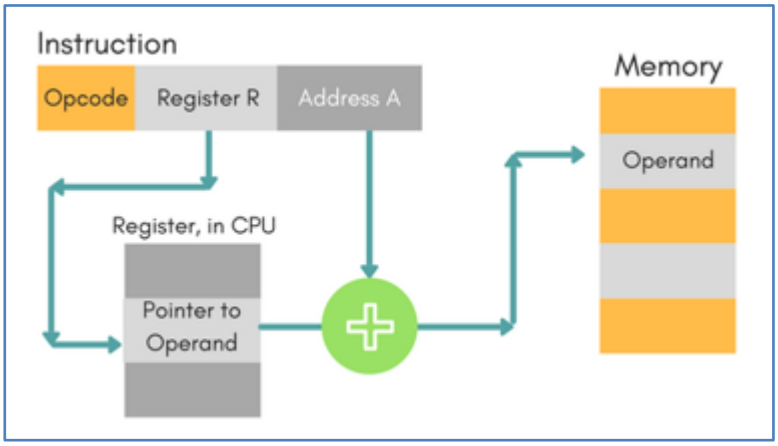
In this, the address field of instruction gives the address where the effective address is stored in memory. This slows down the execution, as this includes multiple memory lookups to find the operand.



(g) Displacement Addressing Mode

In this the contents of the indexed register is added to the Address part of the instruction, to obtain the effective address of operand.

EA = A + (R), In this the address field holds two values, A(which is the base value) and R(that holds the displacement), or vice versa.



(h) Relative Addressing Mode

It is a version of Displacement addressing mode.

In this the contents of PC(Program Counter) is added to address part of instruction to obtain the effective address.

EA = A + (PC), where EA is effective address and PC is program counter.

The operand is a cell away from the current cell(the one pointed to by PC)

(i) Base Register Addressing Mode

It is again a version of Displacement addressing mode. This can be defined as **EA = A + (R)**, where A is displacement and R holds pointer to base address.

(j) Stack Addressing Mode

In this mode, operand is at the top of the stack. For example: **ADD**, this instruction will POP top two items from the stack, add them, and will then PUSH the result to the top of the stack.

3.0 CODES AND THEIR CONVERSIONS

Introductions

Digital circuits use binary signals but are required to handle data which may be alphabetic, numeric, or special characters.Hence, the signalsthat are available in some other form other than binary have to be converted intosuitable binary form before digital circuits can process them further.This meansthat in whatever format the information may be available it must be converted into binary format.
To achieve this, a process of coding is required where each letter, special character, or numeral is coded in a unique combination of 0s and 1s using a coding scheme knownas ***code***.

In digital systems a variety of codes are used to serve different purposes, such as *data entry, arithmetic operation, error detection and correction*, etc. Selection of a particular codedepends on the requirement.

Even in a *single digital system,* a number of different codesmay be used for different operations and it may even be necessary to convert data from one type of code to another. For conversion of data, ***code converter circuits*** are required.

3.1 CODES

Computers and other digital circuits process data in binary format. Various binary codes are used to represent data that may be numeric, alphabetic or special characters.

Codes can be broadly classified into five groups, viz. (i) Weighted Binary Codes,(ii) Non weighted Codes, (iii) Error-detection Codes, (iv) Error-correcting Codes, and(v) Alphanumeric Codes.

3.1.1 Weighted Binary Codes

If each position of a number represents a specific weight then the coding scheme is calledweighted binary code. In such coding, the bits are multiplied by their corresponding individual weight, and then the sum of these weighted bits gives the equivalent decimal digit.

(a) BCD Code or 8421 Code

The full form of BCD is 'Binary-Coded Decimal.' Since this is a coding scheme relating decimal and binary numbers, four bits are required to code each decimal number. For example, $(35)_{10}$ is represented as

0011 0101 using BCD code, rather than (100011). From the example, it is clear that it requires more number of bits to code a decimal number using BCD code than using the ***straight binary code***. However, in spite of this disadvantage it is convenient to use BCD code for input and output operations in digital systems.

The code is also known as ***8-4-2-1 code***. This is because 8, 4, 2, and 1 are the weights of the four bits of the BCD code. The weight of the LSB is 2^0 or 1 that of the next higher order bit is 2^1 or 2, that of the next higher order bit is 2^2 or 4, and that of the MSB is 2^3 or 8. Therefore, this is a weighted code and arithmetic operations can be performed using this code.

The bit assignment 0101, for example, can be interpreted by the weights to represent the decimal digit 5 because $0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 5$. Since four binary bits are used, the maximum decimal equivalent that may be coded is 15(i.e., 1111). However, the maximum decimal digit available is 9_{10}. Hence the binary codes 1010, 1011, 1100, 1101, 1110, 1111, representing 10, 11, 12, 13, 14, and 15 in decimal are never being used in BCD code. So these six codes are called forbidden codes and the group of these codes is called the forbidden group in BCD code.

Example 3.1

Give the BCD equivalent for the decimal number 589.

Solution. The decimal number is 589

BCD code is 0101 1000 1001

Hence, $(589)_{10} = (010110001001)_{BCD}$

Example 3.2

Give the BCD equivalent for the decimal number 69.27.

Solution. The decimal number 69.27

BCD code is 0110 1001 0010 0111

Hence, $(69.27)_{10} = (01101001.00100111)$

(b) 84-2-1 Code

It is also possible to assign negativeweights to decimal code, as shown by the 84-2-1 code.

In this case, the bit combination 0101 is interpreted as the decimal digit 3, as obtained from 0 × 8 + 1 × 4 + 0 × (−2) + 1 × (−1) = 3.
This is a self-complementary code, that is, the 9's complement of the decimal number is obtained just by changing the 1s to 0s and 0s to 1s, or in effect by getting the 1's complement of the corresponding number. For example, if we change the 1s to 0s and 0s to 1s in the previous example we have 1010, which is interpreted as decimal 6, as obtained from 1 × 8 + 0 × 4 + 1 × (−2) + 0 × (−1)= 6. We know that 6 is the 9's complement of 3.

(c) 2421 Code

Another weighted code is 2421 code. The weights assigned to the four digits are 2, 4, 2, and 1.

The 2421 code is the same as that in BCD from 0 to 4; however, it varies from 5 to 9.

For example, in this case the bit combination 0100 represents decimal 4; whereas the bit combination 1101 is interpreted as the decimal 7, as obtained from 2 × 1 + 1 × 4+ 0 × 2 + 1 × 1 = 7.

This is also a self-complementary code, that is, the 9's complement of the decimal number is obtained by changing the 1s to 0s and 0s to 1s.

Table 3.1: Binary Codes for decimal digits

Decimal digit	*(BCD) 8421*	*84-2-1*	*2421*	*Excess-3*
0	0000	0000	0000	0011
1	0001	0111	0001	0100
2	0010	0110	0010	0101
3	0011	0101	0011	0110
4	0100	0100	0100	0111
5	0101	1011	1011	1000
6	0110	1010	1100	1001
7	0111	1001	1101	1010
8	1000	1000	1110	1011
9	1001	1111	1111	1100

3.1.2 Nonweighted Codes

These codes are not positionally weighted. It basically means that each position of the binary number is not assigned a fixed value. ***Excess-3 codes and Gray codes*** are suchnon-weighted codes.

(a) Excess-3 Code

A decimal code that has been used in some old computers is Excess-3 code. This is a nonweighted code. This code assignment is obtained from the corresponding value of 4-bit binary code after ***adding 3 to the given decimal digit***.

Here the maximum value may be 1100_2 . Since the maximum decimal digit is 9 we have to add 3 to 9 and then get the BCD equivalent.

Like 84-2-1 and 2421 codes Excess-3 is also a self-complementary code, that is, the 9's complement of the decimal number is obtained by changing the 1s to 0s and 0s to 1s. This self-complementary property of the code helps considerably in performing subtraction operation in digital systems.

Example 3.3:

Convert $(367)_{10}$into its Excess-3 code.

Solution. The decimal number is 3 6 7

Add 3 to each bit +3 +3 +3

Sum 6 9 10

Converting the above sum into 4-bit binary equivalent, we have a 4-bit binary equivalent of 0110 1001 1010

Hence, the Excess-3 code for $(367)_{10}$= 0110 1001 1010

Example 3.4

Convert $(58.43)_{10}$into its Excess-3 code.

Solution. The decimal number is 5 8 4 3

Add 3 to each bit +3 +3 +3 +3

Sum 8 11 7 6

Converting the above sum into 4-bit binary equivalent, we have a 4-bit binary equivalent of 1000 1011 0111 0110

Hence, the Excess-3 code for $(58.43)_{10}$ = (1000 1011.0111 0110)

(b) Gray Code

Gray code belongs to a class of code known as minimum change code, in which a number changes by only one bit as it proceeds from one number to the next.

This code finds extensive use for shaft encoders, in some types of Analog-to-digital converters, etc.

The Gray code is not a weighted code.

Algorithm or the Procedures for the Conversion of a Binary Number into Gray Code

Any binary number can be converted into equivalent Gray code by the following steps:
(i) The MSB of the Gray code is the same as the MSB of the binary number;
(ii) The second bit next to the MSB of the Gray code equals the Ex-OR of the MSB and second bit of the binary number; it will be 0 if there are same binary bits or it will be 1 for different binary bits;
(iii) the third bit for Gray code equals the exclusive-OR of the second and third bits of the binary number, and similarly all the next lower order bits follow the samemechanism.

Example 3.5. Convert $(101011)_2$
Solution.

Step 1. The MSB of the Gray code is the same as the MSB of the binary number.

1	0	1	0	1	1	Binary
↓						
1						Gray

Step 2. Perform the ex-OR between the MSB and the second bit of the binary. The result is 1, which is the second bit of the Gray code.

1 ⊕	0	1	0	1	1	Binary
	↓					
1	1					Gray

Step 3. Perform the ex-OR between the second and the third bits of the binary. The result is 1, which is the third bit of the Gray code.

1	0 ⊕	1	0	1	1	Binary
		↓				
1	1	1				Gray

Step 4. Perform the ex-OR between the third and the fourth bits of the binary. The result is 1, which is the fourth bit of the Gray code.

1	0	1 ⊕	0	1	1	Binary
			↓			
1	1	1	1			Gray

After completing the conversion, the Gray code of binary 101011 is 111110.

Example 3.6. Convert $(564)_{10}$ to Gray code

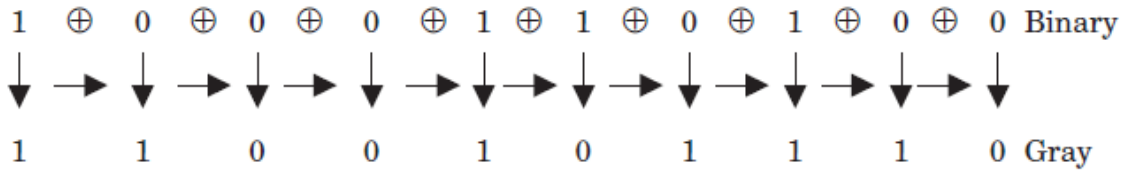
Solution.

Step 1. Convert the decimal 564 into equivalent binary.

Decimal number 564

Binary number 1000110100

Step 2. Convert the binary number into equivalent Gray code.



Algorithm or the Procedures for the Conversion of Gray Code into a Binary Number

Any Gray code can be converted into an equivalent binary number by the following steps:

(i) The MSB of the binary number is the same as the MSB of the Gray code;

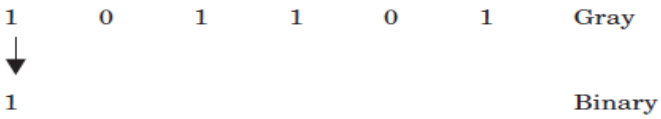
(ii) The second bit next to the MSB of the binary number equals the Ex-OR of the MSB of the binary number and second bit of the Gray code; it will be 0 if there are same binary bits or it will be 1 for different binary bits;

(iii) The third bit for the binary number equals the exclusive-OR of the second bit of the binary number and third bit of the Gray code, and similarly all the next lower orderbits follow the same mechanism.

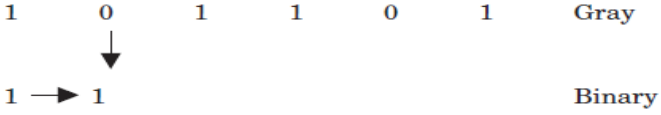
Example 3.7. Convert the Gray code 101101 into a binary number.

Solution.

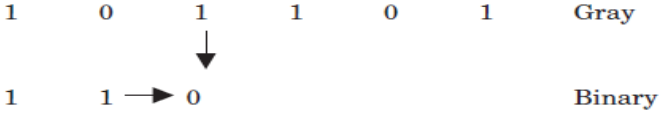
Step 1. The MSB of the binary number is the same as the MSB of the Gray code.



Step 2. Perform the ex-OR between the MSB of the binary number and the second bit of the Gray code. The result is 1, which is the second bit of the binary number.



Step 3. Perform the ex-OR between the second bit of the binary number and the third bit of the Gray code. The result is 0, which is the third bit of the binary number.



After completing the conversion, the binary number of the Gray code 101101 is**110110.**

Table 3.2: Binary and Gray codes

Decimal numbers	*Binary code*	*Gray code*
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

3.1.3 Error-detection Codes

(a) Parity Bit Coding Technique

One of the characteristics of binary data transmission is that they are easily susceptible to error in the form of noise.

Binary information may be transmitted through some form of communication medium such as wires or radio waves or fiber optic cables, etc. Any external noise introduced into a physical communication medium changes bit values from 0 to 1 or vice versa.

An error- detection code can be used to detect errors during transmission. The detected error cannot be corrected, but its presence is indicated.

A ***parity bit*** is an extra bit included with a message to make the total number of 1s either odd or even. A message of four bits and a parity bit, P, are shown in Table 3.3. In (a), P is chosen so that the sum of all 1s is odd (including the parity bit). In (b), P is chosen so that the sum of all 1s is even (including the parity bit).

In the sending end, the message (in this case the first four bits) is applied to a "parity generation" circuit where the required P bit is generated. The message, including the parity bit, is transferred to its destination. In thereceiving end, all the incoming bits (in this case five) are applied to a "parity check" circuit to check the proper parity adopted.

An error is detected if the checked parity does not correspond to the adopted one. The parity method detects the presence of one, three, five, or any odd combination of errors. An even combination of errors is undetectable since an even number of errors will not change the parity of the bits.

The parity bit may be included with the message bits either on the MSB or on the LSB side. Hence, in such cases, some other coding scheme is to be adopted. Such a coding technique is Check Sums.

Table 3.3:Parity Bits

(a) Message	*P (odd)*	*(b) Message*	*P (even)*
0000	1	0000	0
0001	0	0001	1
0010	0	0010	1
0011	1	0011	0
0100	0	0100	1
0101	1	0101	0
0110	1	0110	0
0111	0	0111	1
1000	0	1000	1
1001	1	1001	0
1010	1	1010	0
1011	0	1011	1
1100	1	1100	0
1101	0	1101	1
1110	0	1110	1
1111	1	1111	0

(b) Check Sums

The parity bit technique fails for double errors, hence we use the Check Sums method in such case. Initially any word A 10010011 is transmitted; next, another word B 01110110 is transmitted. The binary digits in the two words are added and the sum obtained is retained in the transmitter. Then any other word C is transmitted and added to the previous sum retained in the transmitter and

the new sum is now retained. In a similar manner, each word is added to the previous sum already retained; after transmitting all the words, the final sum, which is called the Check Sum, is also transmitted. The same operation is done at the receiving end and the final sum, which has been obtained here, is being checked

against the transmitted Check Sum. There is no error if the two sums are equal.

3.1.4 Error-correcting Codes

We discussed in section 3.1.3 two coding techniques that may be used in transmission to detect errors. But, unfortunately, those discussed above are not capable of correcting the errors. For correction of errors we now discuss a code called the Hamming code.

(a) Hamming Code

R. W. Hamming had developed this coding where one or more parity bits are added to a data character methodically in order to detect and correct errors. The number of bits changed from one code word to another is known as Hamming distance.

Let us consider $A_i$and $A_j$to be any two code words in any particular block code. Nowthe Hamming distance d_{ij}between the two vectors $A_i$and $A_j$is defined by the number ofcomponents in which they differ. Assuming that $d_{ij}$is determined for each pair of code words,the minimum value of $d_{ij}$is called the minimum Hamming distance, d_{min}

For example,

A_i	1	0	0	1	0	1	1
	↓		↓			↓	
A_j	0	0	1	1	0	0	1

Here, these code words differ in the MSB and in the third and sixth bit positions from the left. Hence, d_{ij} is 3.

From Hamming's analysis of code distances, some important properties have been derived:

(*i*) For detection of a single error d_{min} should be at least two.

(*ii*) For single error correction, d_{min} should be at least three, since the number of errors, $E \leq [(d_{min} - 1)/2]$.

(*iii*) Greater values of d_{min} will provide detection and/or correction of more number of errors.

The 7-bit Hamming (7, 4) code word $h_1\ h_2\ h_3\ h_4\ h_5\ h_6\ h_7$ associated with a 4-bit binary number $b_3\ b_2\ b_1\ b_0$ is:

$$h_1 = b_3 \oplus b_2 \oplus b_0$$
$$h_2 = b_3 \oplus b_1 \oplus b_0$$
$$h_4 = b_2 \oplus b_1 \oplus b_0$$
$$h_3 = b_3$$
$$h_5 = b_2$$
$$h_6 = b_1$$
$$h_7 = b_0$$

Bits h_1, h_2, and h_4 produce even parity bits for the bit fields $b_3\ b_2\ b_0$, $b_3\ b_1\ b_0$, and $b_2\ b_1\ b_0$ respectively. Generally the parity bits (h_1, h_2, h_4, h_8, h_{16}...) are located in the positions corresponding to ascending powers of two (*i.e.*, 2^0, 2^1, 2^2, 2^3, 2^4... = 1, 2, 4, 8,16...).

The h_1 parity bit has a 1 in the LSB position of its binary representation. Therefore it can check all the bit positions, including those that have 1s in the LSB position in the binary representation (*i.e.*, h_1, h_3, h_5, and h_7). The binary representation of h_2 has a 1 in the middle bit position. Therefore it can check all the bit positions, including those that have 1s in the middle bit position in the binary representation (*i.e.*, h_2, h_3, h_6, and h_7). The h_4 parity bit has a 1 in the MSB position of its binary representation. Therefore it can check all the bit positions, including those that have 1s in the MSB position in the binary representation (*i.e.*, h_4, h_5, h_6, and h_7).

To decode a Hamming code, checking needs to be done for odd parity over the bit fields in which even parity was previously established. For example, a single bit error is indicated by a nonzero parity word $a_4\ a_2\ a_1$, where

$$a_1 = h_1 \oplus h_3 \oplus h_5 \oplus h_7$$

$$a_2 = h_2 \oplus h_3 \oplus h_6 \oplus h_7$$
$$a_4 = h_4 \oplus h_5 \oplus h_6 \oplus h_7$$

If $a_4\ a_2\ a_1 = 000$, we conclude there is no error in the Hamming code. On the other hand, if it has a nonzero value, it indicates the bit position in error. For example, if $a_4\ a_2\ a_1 = 110$, then bit 6 is in error. To correct this error, bit 6 has to be complemented.

Example 3.8. Encode data bits 0110 into a 7-bit even parity Hamming code.

Solution. Given $b_3\ b_2\ b_1\ b_0 = 0\ 1\ 1\ 0$

Therefore,
$$h_1 = b_3 \oplus b_2 \oplus b_0 = 0 \oplus 1 \oplus 0 = 1$$
$$h_2 = b_3 \oplus b_1 \oplus b_0 = 0 \oplus 1 \oplus 0 = 1$$
$$h_4 = b_2 \oplus b_1 \oplus b_0 = 1 \oplus 1 \oplus 0 = 0$$
$$h_3 = b_3 = 0$$
$$h_5 = b_2 = 1$$
$$h_6 = b_1 = 1$$
$$h_7 = b_0 = 0$$

h_1	h_2	h_3	h_4	h_5	h_6	h_7
1	1	0	0	1	1	0

Example 3.9: A 7-bit Hamming code is received as 0110110. What is its correct code?

Solution.

h_1	h_2	h_3	h_4	h_5	h_6	h_7
0	1	1	0	1	1	0

Now, to find the error,

$$a_1 = h_1 \oplus h_3 \oplus h_5 \oplus h_7 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$$
$$a_2 = h_2 \oplus h_3 \oplus h_6 \oplus h_7 = 1 \oplus 1 \oplus 1 \oplus 0 = 1$$
$$a_4 = h_4 \oplus h_5 \oplus h_6 \oplus h_7 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$$

Thus, $a_4\ a_2\ a_1$ = 010. Therefore, bit 2 is in error and the corrected code can be obtained by complementing the second bit in the received as 00 10110.

3.1.5 Alphanumeric Codes

Many applications of the computer require not only handling of numbers, but also of letters. To represent letters it is necessary to have a binary code for the alphabet.

In addition, the same binary code must represent the decimal numbers and some other special characters.

*An **alphanumeric code** is a binary code of a group of elements consisting of **ten decimal digits, the 26 letters of the alphabet (both in uppercase and lowercase), and a certain number of special symbols such as #, /, &, %, etc.***

The total number of elements in an alphanumeric code is greater than 36. Therefore it must be coded with a minimum number of 6 bits (2^6 = 64, but 2^5 = 32 is insufficient).

3.1.6 ASCII

The full form of ASCII (pronounced "as-kee") is "American Standard Code for Information Interchange," used in most microcomputers.

It is actually a *7-bit code*, where a character is represented with seven bits. The character is stored as one byte with one bit remaining unused. But often the extra bit is used to extend the ASCII to represent an additional 128 characters.

3.1.7 EBCDIC

EBCDIC The full form of EBCDIC is "Extended Binary Coded Decimal Interchange Code." It is also an alphanumeric code generally used in IBM equipment and in large computers for communicating alphanumeric data.

For the different alphanumeric characters the code grouping in this code is different from the ASCII code. It is actually an 8-bit code and a ninth bit is added as the parity bit.

4.0 BOOLEAN ALGEBRA

The digital circuitry in digital computers and other digital systems is designed, and its behavior is analyzed, with the use of a mathematical discipline known as **Boolean algebra**.

The name is in honor of an English mathematician ***George Boole***, who proposed the basic principles of this algebra in 1854 in his treatise, *An Investigation of the Laws of Thought on Which to Found the* ***Mathematical Theories of Logic and Probabilities***.

In 1938, ***Claude Shannon***, a research assistant in the Electrical Engineering Department at M.I.T., suggested that Boolean algebra could be used to solve problems in ***relay-switching circuit design*** [SHAN38]. Shannon's techniques were subsequently used in the analysis and design of electronic digital circuits.

Boolean algebra turns out to be a convenient tool in two areas:

(a) **Analysis:** It is an economical way of describing the function of digital circuitry.

(b) Design: Given a desired function, Boolean algebra can be applied to develop a simplified implementation of that function.

* Boolean Algebra makes use of logical variables and operations.

Thus, a variable may take on the value 1 (TRUE) or 0 (FALSE). The basic logical operations are AND, OR, and NOT, which are symbolically represented by dot, plus sign, and overbar.

A AND B = A . B.
A OR B = A + B
NOT A = $\overline{A}$

Other operations are NOR, NAND, XOR.

Truth tables can be constructed for all the basic and the other operations listed above. A ***truth table*** is a table showing the outputs for all possible combinations of inputs to a logic gate or circuit. Truth table can be constructed for Boolean operators with two inputs variables and same can be extended for more than two variables as shown in Table 4.1(a) and 4.1(b).

4.1 Gates

The fundamental building block of all digital logic circuits is the gate. Logical functions are implemented by the interconnection of gates.

A gate is an electronic circuit that produces an output signal, which is a simple Boolean operation on its input signals. The basic gates used in digital logic are AND, OR, NOT, NAND, NOR, and XOR. Figure 4.1 depicts these six gates. Each gate is defined in three ways: graphic symbol, algebraic notation, and truth table.

Table 4.1: (a) and (b): Truth tables for Boolean operators

(a) Boolean Operators of Two Input Variables

P	Q	NOT P ($\overline{P}$)	P AND Q ($P \cdot Q$)	P OR Q ($P + Q$)	P NAND Q ($\overline{P \cdot Q}$)	P NOR Q ($\overline{P + Q}$)	P XOR Q ($P \oplus Q$)
0	0	1	0	0	1	1	0
0	1	1	0	1	1	0	1
1	0	0	0	1	1	0	1
1	1	0	1	1	0	0	0

(b) Boolean Operators Extended to More than Two Inputs (A, B, . . .)

Operation	Expression	Output = 1 if
AND	$A \cdot B \cdot \ldots$	All of the set {A, B, ...} are 1.
OR	$A + B + \ldots$	Any of the set {A, B, ...} are 1.
NAND	$\overline{A \cdot B \cdot \ldots}$	Any of the set {A, B, ...} are 0.
NOR	$\overline{A + B + \ldots}$	All of the set {A, B, ...} are 0.
XOR	$A \oplus B \oplus \ldots$	The set {A, B, ...} contains an odd number of ones.

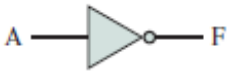
Name	Graphical Symbol	Algebraic Function	Truth Table
AND		$F = A \bullet B$ or $F = AB$	A B | F 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1
OR		$F = A + B$	A B | F 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1
NOT		$F = \overline{A}$ or $F = A'$	A | F 0 | 1 1 | 0
NAND		$F = \overline{AB}$	A B | F 0 0 | 1 0 1 | 1 1 0 | 1 1 1 | 0
NOR		$F = \overline{A + B}$	A B | F 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 0
XOR		$F = A \oplus B$	A B | F 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 0

Figure 4.1: Basic Logic Gates